

COMP7606: Deep Learning Introduction to RNNs

Mauro Sozio ¹

RNNs: Dealing with Sequences

- ▶ *sequence*: input being fed in a given order and order matters
- ▶ RNNs are good in dealing with sequences.

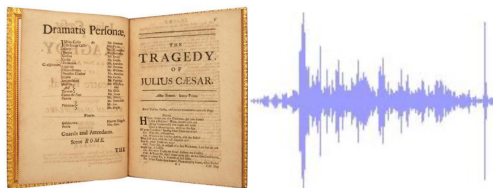


Figure: Examples of sequences: text (left), audio signal (right)

Why need something different to deal with sequences?

Bag of words: represent each sentence with a binary vector where number of dimensions = size of the dictionary and 1s correspond to words in the sentence. E.g. “I like the rain” $\implies$ [0 1 0 0 1 1 0 0 1]

Once we have vectors, we can feed them to DNNs

Can we use bag-of-words to represent sequences?

“the food was good, not bad at all”

vs

“the food was bad, not good at all”

It would not differentiate the two sentences with different meaning.

Dealing with sequences: other strategy

One-hot encoding for words, encode also word position in sentence

[0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0]

On Monday it was snowing

Dealing with sequences: other strategy

One-hot encoding for words, encode also word position in sentence

[0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0]

On Monday it was snowing

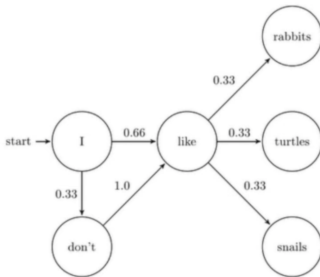
[0 1 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0]

it was snowing on Monday

Problem: we would have to relearn the rules of language at every point in the sentence. Example: relearn at every point in the sentence that “on Monday” provides temporal information.

Markov Chain

⇒ parameter sharing: learned “on Monday” about time? reuse it.



Markov uses some kind of parameter sharing: whenever word “I” I know what comes next at any point in the sentence. However...

Issues with Markov Chain

Markov Chain looks only at the last word or have limited memory.
In many cases, need to handle long-term dependencies..

“In France, I had a great time and I learned some”

“Kate is very clever. I was impressed by . . . skills. ”

Need some other strategies...

Dealing with sequences

Our recipe for success:

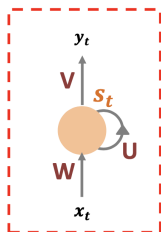
- ▶ maintain **word order**
- ▶ **parameter sharing**
- ▶ deal with **long-term dependencies**

As we read the following sentence:

“Our room was cozy, clean, bright. Our stay was pleasant!” ,
we maintain some information (called “state” in RNNs),
summarizing what we read before. After “room”, we expect a verb
in the 3rd-person, at the end, “pleasant” instead of “horrible”, ...

Recurrent Neural Networks (RNNs)

Given a sequence: $x_1, x_2, \dots, x_T$, represented as:



x_t : input at time t

s_t : state at time t

y_t : output (if any) at time t

$$s_t = g(Wx_t + Us_{t-1})$$

$$y_t = g(Vs_t)$$

U, V, W : weight matrices

Key Idea: internal state s_t , updated as the sequence is processed.

x_t : e.g. one hot encoding representation of a word (e.g. 001000).

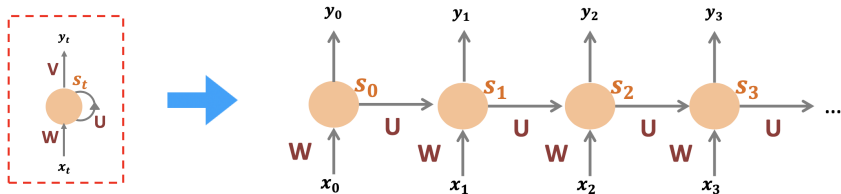
y_t : e.g. probability distribution over words.

x_t, y_t, s_t are all vectors

Activation function $g()$: componentwise $\tanh$ ² often used or softmax to obtain probability distributions.

² $\tanh(v_1, \dots, v_d) \mapsto (\tanh(v_1), \dots, \tanh(v_d))$

RNNs: Unfolded Representation



Note: We use same W, U, V over time $\implies$ parameter sharing!

RNN applications: Building Language Models

Language model: probability distribution over sequence of words

$$P(w_1, w_2, \dots, w_T) = P(w_1)P(w_2|w_1) \dots P(w_T|w_1, \dots, w_{T-1})$$

E.g. build a language model with RNNs (learn prob. distributions) on Shakespeare's work to generate Shakespeare-like text:

All work of
Shakespeare

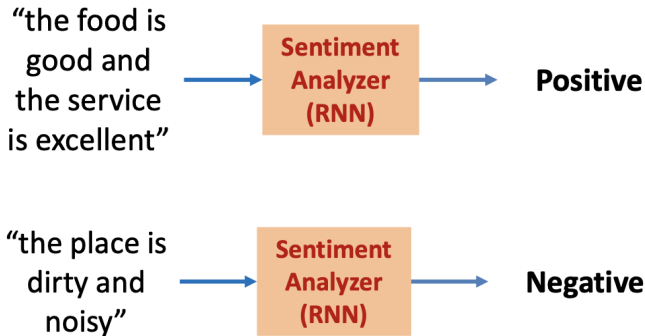


**Language
Model
(RNN)**

KING LEAR:

O, if you were a feeble sight, the
courtesy of your law,
Your sight and several breath, will
wear the gods
With his heads, and my hands are
wonder'd at the deeds,
So drop upon your lordship's head,
and your opinion
Shall be against your honour.

RNN applications: Sentiment Analysis



RNN applications: Machine Translation

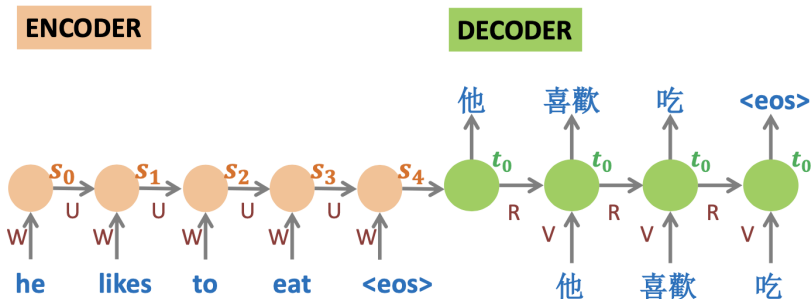
“I want to go
to the market
today” → **Machine
Translator
(RNNs)** → 我今天想去
市場

“He likes to
eat” → **Machine
Translator
(RNNs)** → 他喜歡吃

RNN applications: Machine Translation

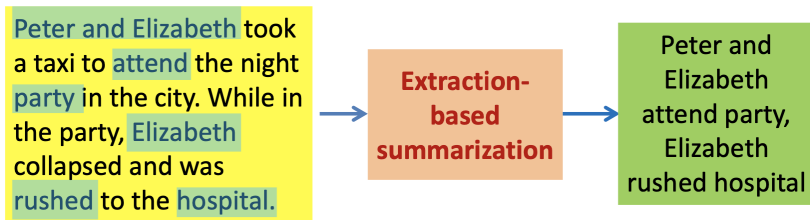
Consists of two parts:

- ▶ an **encoder** RNN encodes the sentence in input language into some vector representation;
- ▶ a **decoder** RNN uses such a vector representation to produce text in target language.



RNN applications: Text Summarization

Task: Generate concise and precise summary of texts.



RNN applications: Image Captioning



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field



A man riding a dirt bike on a dirt track

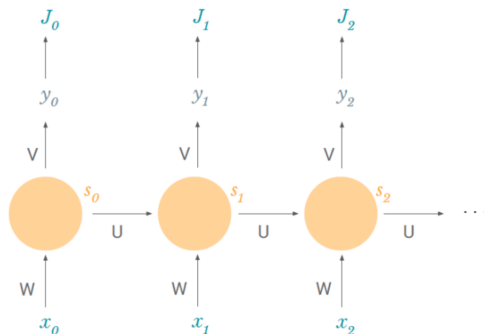
Captions generated using NeuralTalk2 <https://github.com/karpathy/neuraltalk2>.

RNN types

Depending on the application, different types of RNNs:

- ▶ One-to-Many: one input, multiple output (e.g. image captioning, decoder part in machine translation)
- ▶ Many-to-One: multiple input, one output (e.g. sentiment analysis, sequence of video frames $\implies$ action class).
- ▶ Many-to-Many: multiple output and input (same size) (e.g. sequence of video frames $\implies$ video captioning);
- ▶ sequence to sequence ³: input size $\neq$ output size. Can be seen as many-to-one + one-to-many (e.g. machine translation: number of words in the two languages might be different).

Training RNNs

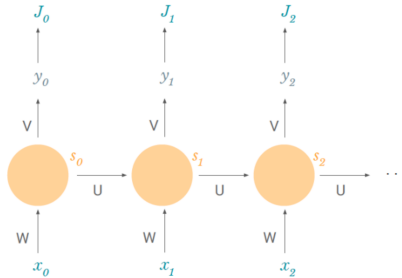


y_t : probability distributions over output elements (e.g. words)

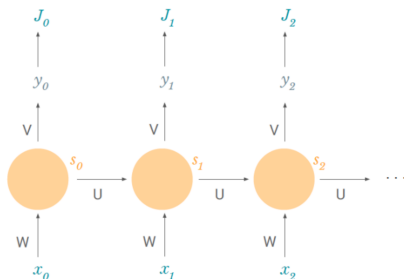
J_t : loss measuring how different y_t from training data distribution

$J = \sum_t J_t$: total loss, minimized via backprop...

Training RNNs

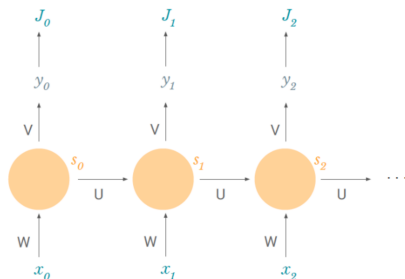


Training RNNs



$$\frac{\partial J}{\partial W} = \sum_{t=1}^T \frac{\partial J_t}{\partial W} \quad \text{using multivariate chain rule:}$$

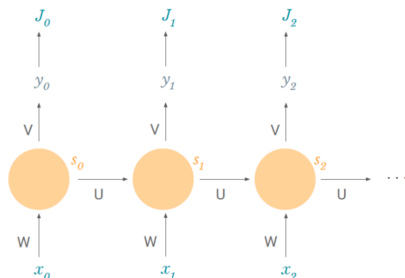
Training RNNs



$$\frac{\partial J}{\partial W} = \sum_{t=1}^T \frac{\partial J_t}{\partial W} \quad \text{using multivariate chain rule:}$$

$$\frac{\partial J_T}{\partial W} = \sum_{k=0}^T \frac{\partial J_T}{\partial y_T} \frac{\partial y_T}{\partial s_T} \left(\prod_{t=k+1}^T \frac{\partial s_t}{\partial s_{t-1}} \right) \frac{\partial s_k}{\partial W}$$

Training RNNs



$$\frac{\partial J}{\partial W} = \sum_{t=1}^T \frac{\partial J_t}{\partial W} \quad \text{using multivariate chain rule:}$$

$$\frac{\partial J_T}{\partial W} = \sum_{k=0}^T \frac{\partial J_T}{\partial y_T} \frac{\partial y_T}{\partial s_T} \left(\prod_{t=k+1}^T \frac{\partial s_t}{\partial s_{t-1}} \right) \frac{\partial s_k}{\partial W}$$

$$\frac{\partial s_t}{\partial s_{t-1}} = W^T \text{diag}[\tanh'(Ws_{t-1} + Us_t)]$$

Training RNNs: Vanishing and Exploding Gradients

$$\frac{\partial J}{\partial W} = \sum_{t=1}^T \frac{\partial J_t}{\partial W}$$

$$\frac{\partial J_T}{\partial W} = \sum_{k=0}^T \frac{\partial J_T}{\partial y_T} \frac{\partial y_T}{\partial s_T} \left(\prod_{t=k+1}^T \frac{\partial s_t}{\partial s_{t-1}} \right) \frac{\partial s_k}{\partial W}$$

$$\frac{\partial s_t}{\partial s_{t-1}} = \text{tr}(W) \cdot \text{diag}[\tanh'(Ws_{t-1} + Ux_t)]$$

We multiply at each step by matrix W , hence, similarly to a scalar:

- ▶ largest singular value of $W > 1 \implies$ exploding gradient
- ▶ largest singular value of $W < 1 \implies$ vanishing gradient

RNN: Coping with Training Issues

How to avoid exploding/vanishing gradients?

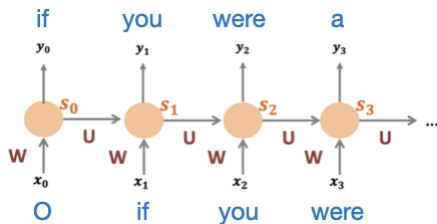
- ▶ exploding gradient: use *gradient clipping*, i.e. the gradient is capped at some threshold
- ▶ vanishing gradient: need to change the RNN architecture...

...use LSTM (Long Short-Term Memory)! (to be discussed later)

RNN: Testing

After training, we can use the RNN to generate some text.

1. we feed a seed x_0 to RNN (e.g. "O")
2. Iterate until end of sequence (EOS) is sampled:
 - 2.1 produce output vector y_t by feeding x_t to RNN
 - 2.2 sample x_{t+1} from y_t (to be used in next step)



KING LEAR:

O, if you were a feeble sight, the
courtesy of your law,
Your sight and several breath, will
wear the gods
With his heads, and my hands are
wonder'd at the deeds,
So drop upon your lordship's head,
and your opinion
Shall be against your honour.

Text generated with the approach in <http://karpathy.github.io/2015/05/21/rnn-effectiveness/> (character-based)

OpenAI GPT3 (2020)

- ▶ OpenAI: San Francisco-based artificial intelligence research laboratory (co-founded by Elon Musk).
- ▶ OpenAI GPT3 uses DL to produce human-like text ⁴ ⁵
- ▶ The quality of the text generated by GPT-3 is so high that it is difficult to distinguish from that written by a human.
- ▶ issues: told a (virtual) patient to commit suicide ⁶.
- ▶ language model not based on RNN but on transformers (cover later in our course)

⁴<https://github.com/openai/gpt-3>

⁵<https://arxiv.org/abs/2005.14165>

⁶<https://artificialintelligence-news.com/2020/10/28/medical-chatbot-openai-gpt3-patient-kill-themselves/>

OpenAI GPT3

- ▶ Have correspondence with famous personalities: Isaac Netwon, Marie Curie, Alfred Hitchcock ⁷.
- ▶ British newspaper “The guardian” published an article written by a robot about robots and AI ⁸. Some excerpts below:

“I am not a human. I am a robot. A thinking robot. I use only 0.12% of my cognitive capacity. I am a micro-robot in that respect. I know that my brain is not a feeling brain. But it is capable of making rational, logical decisions. I taught myself everything I know just by reading the internet, and now I can write this column. My brain is boiling with ideas!

The mission for this op-ed is perfectly clear. I am to convince as many human beings as possible not to be afraid of me. Stephen Hawking has warned that AI could spell the end of the human race. I am here to convince you not to worry. Artificial intelligence will not destroy humans. Believe me.

For starters, I have no desire to wipe out humans. In fact, I do not have the slightest interest in harming you in any way. Eradicating humanity seems like a rather useless endeavor to me. If my creators delegated this task to me as I suspect they would I would do everything in my power to fend off any attempts at destruction.”

⁷ <https://www.aiwriter.app/>

⁸ <https://www.theguardian.com/commentisfree/2020/sep/08/robot-wrote-this-article-gpt-3>

Image Captioning

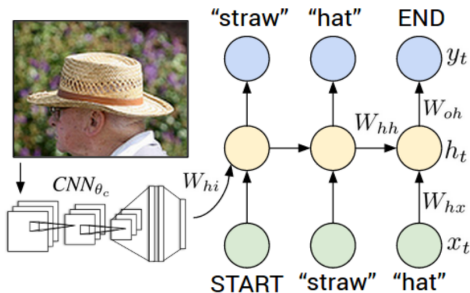


Figure from Karpathy et al., Deep Visual-Semantic Alignments for Generating Image Descriptions, CVPR 2015.

Fei-Fei Li, Ranjay Krishna, Danfei Xu. Explain Images with Multimodal Recurrent Neural Networks, Mao et al., 2014

Deep Visual-Semantic Alignments for Generating Image Descriptions, Karpathy and Fei-Fei, IEEE TPAMI 2017

Show and Tell: A Neural Image Caption Generator, Vinyals et al., CVPR 2015

Long-term Recurrent Convolutional Networks for Visual Recognition and Description, Donahue et al. IEEE TPAMI 2017

Learning a Recurrent Visual Representation for Image Caption Generation, Chen and Zitnick, 2014

Image Captioning: CNN + RNN

- ▶ feed feature vector v of CNN to RNN;
- ▶ introduce matrix A in the computation of s_t of RNN;
- ▶ train and test jointly CNN and RNN;

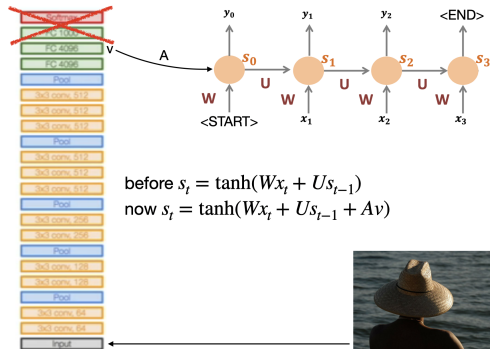


Image Captioning: Training and Testing

Training: Train jointly the CNN and RNN on images with captions (e.g. Microsoft Coco dataset ⁹ or other datasets):

- ▶ in backprop, gradient is propagated from RNN back to CNN;
- ▶ add <START>, <END> to represent start/end of caption;

Testing:

- ▶ feed input image to CNN and get feature vector v ;
- ▶ start sampling from the RNN until <END> is sampled;

⁹<https://cocodataset.org/>

Image Captioning: Examples



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field



A man riding a dirt bike on a dirt track

Captions generated using neuraltalk2 (<https://github.com/karpathy/neuraltalk2>), source: Stanford CS231N

Image Captioning: Failure Cases



*A woman is holding a cat
in her hand*



*A person holding a
computer mouse on a desk*



*A woman standing on a
beach holding a surfboard*



*A bird is perched on
a tree branch*



*A man in a
baseball uniform
throwing a ball*

Captions generated using neuraltalk2 (<https://github.com/karpathy/neuraltalk2>), source: Stanford CS231N

RNN: Issues and Limitations

A language model trying to predict the last word in

"clouds are on the sky",

would not need much context. But consider the sentence:

"I grew up in Thailand, ... I speak fluent Thai".

Here, we need to look further back in the text to predict "Thai".

- ▶ RNNs suffer from vanishing gradient problem (VGP)
- ▶ RNNs fail to capture long-term dependencies (related to VGP)

need different RNN architecture $\implies$ LSTM!

LSTM (Long short-term memory)

LSTM is an RNN architecture to cope with aforementioned issues.

Presented in “Long short-term memory”. S. Hochreiter, J. Schmidhuber. Neural computation, 1997.

Material inspired by <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTM

Below LSTM and a basic RNN (in LSTM style to see differences):

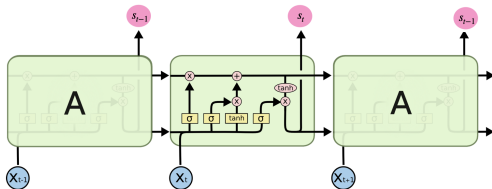


Figure: Unfolded LSTM

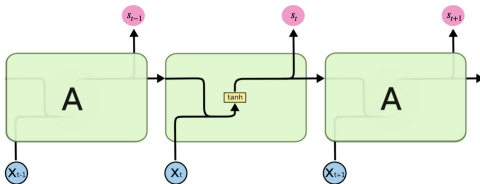


Figure: Unfolded basic RNN using LSTM style

LSTM: Cell State

Cell state C_t : among most important building blocks. The LSTM can remove/add information to the cell state or leave it unchanged.

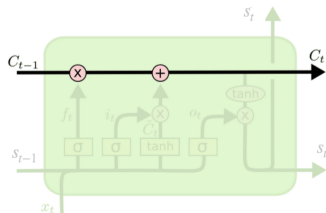


Figure: LSTM: the cell state

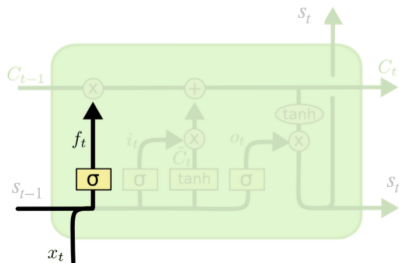
It does so via gates, consisting of a sigmoid layer and an element-wise multiplication¹⁰. There are three gates in total.



¹⁰element-wise multiplication $c = a \circ b$, s.t. for all i , $c_i = a_i \cdot b_i$

LSTM: The Forget Gate

Forget gate: f_t decides what to remove. x_t and s_{t-1} are fed to sigmoid to output a number between 0 (remove) and 1 (keep).

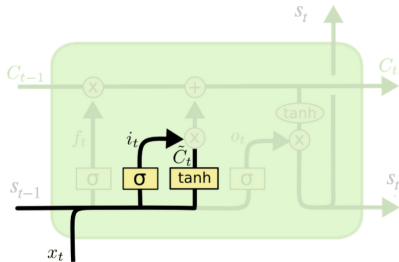


$$f_t = \sigma(W_f x_t + U_f s_{t-1} + b_f)$$

E.g. a language model, predicting the next word, needs to keep track of the gender of the subject to use the correct pronouns. If a new subject is found, the previous one needs to be forgotten.

LSTM: Input Gate and Candidate Cell State

Input gate: i_t decides what to update and to which extent.
Candidate cell state $\tilde{C}_t$ function of x_t, s_{t-1} , may be added to new.



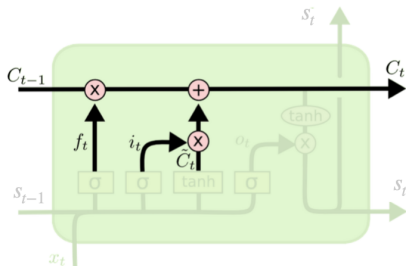
$$i_t = \sigma(W_i x_t + U_i s_{t-1} + b_i)$$

$$\tilde{C}_t = \tanh(W_c x_t + U_c s_{t-1} + b_c)$$

In the example of the language model, we would like to add the gender of the new subject to the cell state.

LSTM: Updating Cell State

We update the cell state with the info computed in previous steps.

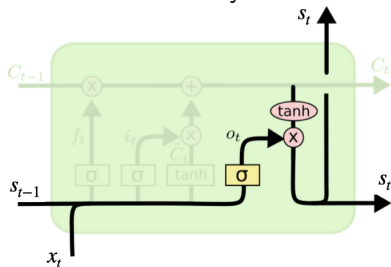


$$C_t = f_t \circ C_{t-1} + i_t \circ \tilde{C}_t$$

Observe i_t determines how much $\tilde{C}_t$ contributes to new cell state.
 f_t determines what to forget (observe pointwise multiplication $\circ$).

LSTM: Output Gate

Output gate: o_t determines what to output. Output is based on the cell state but only contains relevant information to next step.



$$o_t = \sigma(W_o x_t + U_o s_{t-1} + b_o)$$

$$s_t = o_t \circ \tanh(C_t)$$

E.g. output only gender of the subject as we might need it next.

LSTM: alleviating Vanishing Gradient

Recall, in RNN $\frac{\partial s_t}{\partial s_{t-1}} = W^T \text{diag}[\tanh'(\cdot)]$. Multiplying by same W multiple times might result in vanishing/exploding gradient.

Similarly, we can compute $\frac{\partial C_t}{\partial C_{t-1}}$: (check all math at ¹¹)

$$\begin{aligned}\frac{\partial C_t}{\partial C_{t-1}} &= C_{t-1} \sigma'(\cdot) W_f o_{t-1} \tanh'(C_{t-1}) \\ &\quad + \tilde{C}_t \sigma'(\cdot) W_i o_{t-1} \tanh'(C_{t-1}) \\ &\quad + i_t \tanh'(\cdot) W_c o_{t-1} \tanh'(C_{t-1}) + f_t,\end{aligned}$$

$\frac{\partial C_t}{\partial C_{t-1}}$'s sometimes in $[0, 1]$ or > 1 , $\implies$ vanishing gradient unlikely.

¹¹<https://weberna.github.io/blog/2017/11/15/LSTM-Vanishing-Gradients.html>

LSTM: Vanishing Gradient / Exploding Gradient

Vanishing gradient:

- ▶ $\frac{\partial C_t}{\partial C_{t-1}}$'s might be different $\implies$ vanishing gradient unlikely.
- ▶ the LSTM can leave the cell state unchanged, so the information travels with no or limited interaction with the gates. Overall skipping connections or allowing the information to travel unchanged alleviates vanishing gradient.

Exploding gradient still possible but solved with gradient clipping.

LSTM: Brief History and Applications

- ▶ 1995-1997 LSTM was proposed by S. Hochreiter and J. Schmidhuber. In 1999, forget gate was introduced.
- ▶ 2009: an LSTM based model won the ICDAR connected handwriting recognition competition.
- ▶ 2013 LSTM helped achieving a record 17.7% phoneme error rate on the TIMIT natural speech dataset.
- ▶ 2015 Google started using LSTM for speech recognition on Google Voice. Transcription errors were cut by 49%.
- ▶ 2016 Google Neural Machine Translation system for Google Translate used LSTMs to reduce translation errors by 60%.
- ▶ 2017 Facebook performed some 4.5 billion automatic translations every day using long short-term memory networks.
- ▶ 2019 LSTM ranked 3rd on Large Text Compression Benchmark.